

Practica 10: Métodos Para Minimizar Funciones Numéricamente

La minimización y maximización de funciones es una herramienta esencial en las matemáticas aplicadas. Para ello, cada vez que se puede, solemos recurrir a los métodos para encontrar máximos y mínimos de forma analítica. Sin embargo, no siempre son fáciles de aplicar, o peor aún ni siquiera es posible. En esta práctica veremos algunos métodos para minimizar funciones de forma numérica.

A lo largo de la práctica estaremos trabajando con las siguientes dos funciones f_1 y f_2 , las cuales están definidas de $\mathbb{R}^2$ en $\mathbb{R}$. La primera función está dada por la siguiente regla de correspondencia:

$$f_1(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

y el dominio de búsqueda para los mínimos de esta función será $[-5, 5] \times [-5, 5]$.

La segunda función está dada por la siguiente regla de correspondencia:

$$f_2(x, y) = -(y + 47) \sin\left(\sqrt{\left|y + \frac{x}{2} + 47\right|}\right) - x \sin\left(\sqrt{\left|y + \frac{x}{2} + 47\right|}\right)$$

y el dominio de búsqueda para los mínimos de esta función será $[-512, 512] \times [-512, 512]$.

Ejercicio 1: En camino a los métodos numéricos

Paso 1:

Antes de comenzar a programar los métodos es importante definir bien nuestras funciones y sus gradientes. Una recomendación al trabajar con funciones multivariadas es definir que tipo de datos están recibiendo y que tipo de datos están regresando.

Si se fijan, podríamos decir que f_1 y f_2 reciben un vector $x = (x, y)$ o que reciben directamente dos números reales x y y . Esto puede generar problemas a la hora de correr la función en python por lo que siempre es bueno especificar que recibe y que regresa la función. Por ejemplo:

```
def Norma(X: list) -> float:
    x = X[0]
    y = X[1]

    return (x**2 + y**2)**(1/2)
```

Si se fijan esta función recibe una lista x la cual tiene las coordenadas x, y de un vector, calcula la norma del mismo y regresa un número real. Siguiendo este esquema definan f_1 y f_2 de forma que reciban una lista y regresen un flotante.

Paso 2:

Para el primer método que usaremos necesitamos el gradiente de f_1 y f_2 , sin embargo obtener el gradiente de f_2 implica más cuentas de las que podemos hacer ahora. Para evitar eso veamos rápido un método de derivación numérica. Sabemos que f se puede expresar en términos de su serie de Taylor de forma que

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \dots$$

Entonces, si tomamos $x = x_0 + h$, se tiene que $h = x - x_0$ y sustituyendo en la serie de Taylor y truncando a los primeros dos términos tenemos que

$$f(x_0 + h) \approx f(x_0) + f'(x_0)(h)$$

Así, si despejamos $f'(x_0)$ tenemos que

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

Entonces, si $h > 0$, decimos que la expresión anterior es la primera diferencia finita hacia adelante.

Ahora, para reducir un poco el error de nuestra aproximación de la derivada, observemos que también podemos obtener la diferencia finita hacia atrás si $h > 0$ de forma que:

Tomando la aproximación de Taylor si tomamos $x = x_0 - h$ y se tiene que $-h = x - x_0$

$$f(x_0 - h) \approx f(x_0) + f'(x_0)(-h)$$

Así, si despejamos $f'(x_0)$ tenemos que

$$f'(x_0) \approx \frac{f(x_0) - f(x_0 - h)}{h}$$

Entonces tenemos que si sumamos la primera diferencia finita hacia adelante y la primera diferencia finita hacia atrás obtendremos la diferencia finita centrada de la siguiente manera:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}$$

Podemos usar esta misma expresión para calcular las derivadas parciales de una función $f(x, y)$ y así obtener su gradiente. De forma que:

$$\frac{\partial f}{\partial x}(x_0) \approx \frac{f(x_0+h, y) - f(x_0-h, y)}{2h} \text{ y } \frac{\partial f}{\partial y}(y_0) \approx \frac{f(x, y_0+h) - f(x, y_0-h)}{2h}$$

Usa estas expresiones para programar una función que aproxime el gradiente de una función. La función debe recibir una lista X con las coordenadas del punto donde se evaluará, f la función a evaluar y h el tamaño de paso en la diferencia finita centrada. Puedes usar la siguiente base para empezar:

```
def gradf(X: list, f: callable, h: float=0.000001) -> list:
    # Aquí va tu función
```

Pista : hagan dos funciones auxiliares que calculen las formulas de la diferencia finita centrada en cada paso y mandenlas a llamar dentro de la función gradf.

Ejercicio 2: Método Gradiente

Paso 1:

La implementación del método gradiente es muy simple haciendo uso de todo lo que programaron en el Ejercicio 1. El método está dado por la siguiente relación de recurrencia:

$$x_{n+1} = x_n - a f'(x_n), f: R \rightarrow R$$

$$x_{n+1} = x_n - a \nabla f(x_n), f: R^n \rightarrow R$$

$f'(x)$ es la función que calcula el gradiente de f evaluado en el vector x y a es una constante positiva pequeña.

El método crea una sucesión de puntos tales que $f(x_{n+1}) < f(x_n)$

Ahora deben programar una función que reciba como parámetros:

- Un entero positivo n para el número de iteraciones
 - Una lista x_0 que contenga las coordenadas del vector inicial (el cual debe estar en el dominio de búsqueda)
 - a un real positivo
 - f' la función que calcula el gradiente de f . Esta función deberá regresar una lista con los puntos que se generen en cada iteración.
-
- Si la tasa de aprendizaje toma un valor muy pequeño, es necesario un gran número de iteraciones para que el proceso converga

- Si el tasa de aprendizaje es demasiado grande, los cambios en X_{n+1} serán también muy grandes y será difícil encontrar los coeficientes que minimicen la función de coste.

```
import numpy as np
```

Paso 2:

Ahora, prueba tu función con los gradientes numéricos de f_1 y f_2 , con diferentes valores de x_0 en sus respectivos dominios de búsqueda y con diferentes tamaños de paso a . Luego, evalúa tus funciones en el último punto encontrado e intenta encontrar la mejor combinación de x_0 y a de tal forma que esa evaluación verdaderamente sea lo más pequeña posible.

Cuando usamos el gradiente descendiente nos arriesgamos a caer en un mínimo local.

Hay varias mejoras que se han ido realizando a lo largo de los años que ya veremos en otros artículos. Si quieres profundizar puedes leer acerca del gradiente descendiente estocástico, momentum, AdaGrad, RMSProp, Adam, etc.

Ejercicio 3: Un algoritmo evolutivo

Este algoritmo evolutivo tiene un enfoque probabilista. Una primer idea del algoritmo consiste en tomar vecindades de los puntos (x, y) de la forma $[x - a, x + a] \times [y - a, y + a]$ y sobre estas vecindades generar un punto de manera aleatoria y uniforme, i.e. generar un $\hat{x} \sim \text{Uniforme}(x - a, x + a)$ y un $\hat{y} \sim \text{Uniforme}(y - a, y + a)$ y así tomar el punto $(\hat{x}, \hat{y})$. Entonces comparamos los valores de la función a minimizar en (x, y) y en $(\hat{x}, \hat{y})$ y aquel que genere un menor valor será nuestro nuevo punto.

Sin embargo, esto tiene dos problemas. El primero es que en las **funciones que tienen muchos mínimos locales podríamos quedarnos atorados en un mínimo local** y nunca llegar al mínimo global. Para solucionar esto simplemente agregamos un paso más al algoritmo en el que se tira un volado con probabilidad de éxito muy pequeña, m , y con el se decide si generaremos un punto de manera uniforme en la vecindad o si generaremos un punto aleatorio de manera aleatorio en todo el dominio de búsqueda. Así, estaremos asegurándonos de recorrer mayor espacio sin quedarnos atrapados en un mínimo local.

El otro problema es que cuando estamos en puntos muy cercanos a la frontera de nuestra región de búsqueda podemos salirnos sin querer. Para solucionar esto simplemente diremos que aceptamos el punto generado $(\hat{x}, \hat{y})$ si está dentro de los límites de la frontera, de lo contrario generaremos uno nuevo hasta que se cumpla la condición.

Así, el algoritmo quedaría de la siguiente manera:

Programa una función que implemente este algoritmo evolutivo.

La función debe recibir como parámetros

- n : el número de iteraciones
- x_0 : la condición inicial dentro del dominio de búsqueda
- a : el parámetro del tamaño de las vecindades
- m : la probabilidad de mutación (de generar uniformemente e todo el dominio)
- x_limits : los límites para x en el dominio de búsqueda,
- y_limits : los límites para y en el dominio de búsqueda
- f : la función a minimizar

Debe regresar una lista con los puntos generados en cada iteración.

Paso 2:

Ahora, prueba tu función con f_1 y f_2 , con diferentes valores de x_0 en sus respectivos dominios de búsqueda, **con diferentes tamaños de paso a y con diferentes probabilidades de mutación m .**

Luego, evalúa tus funciones en el último punto encontrado e intenta encontrar la mejor combinación de x_0 , a y m de tal forma que esa evaluación verdaderamente sea lo más pequeña posible.